In [278…
```
# Logistic Regression on Mushroom Dataset with Custom Visuals and Insights

# ==========================
# 1. Setup and Preprocessing
# ==========================
!pip install ucimlrepo

from ucimlrepo import fetch_ucirepo
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score, confusion_matri
    roc_curve, auc, log_loss
)
from sklearn.linear_model import LogisticRegression
```

```
Requirement already satisfied: ucimlrepo in /opt/anaconda3/lib/python3.12/si
te-packages (0.0.7)
Requirement already satisfied: pandas>=1.0.0 in /opt/anaconda3/lib/python3.1
2/site-packages (from ucimlrepo) (2.2.2)
Requirement already satisfied: certifi>=2020.12.5 in /opt/anaconda3/lib/pyth
on3.12/site-packages (from ucimlrepo) (2025.1.31)
Requirement already satisfied: numpy>=1.26.0 in /opt/anaconda3/lib/python3.1
2/site-packages (from pandas>=1.0.0->ucimlrepo) (1.26.4)
Requirement already satisfied: python-dateutil>=2.8.2 in /opt/anaconda3/lib/
python3.12/site-packages (from pandas>=1.0.0->ucimlrepo) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /opt/anaconda3/lib/python3.1
2/site-packages (from pandas>=1.0.0->ucimlrepo) (2024.1)
Requirement already satisfied: tzdata>=2022.7 in /opt/anaconda3/lib/python3.
12/site-packages (from pandas>=1.0.0->ucimlrepo) (2023.3)
Requirement already satisfied: six>=1.5 in /opt/anaconda3/lib/python3.12/sit
e-packages (from python-dateutil>=2.8.2->pandas>=1.0.0->ucimlrepo) (1.16.0)
```

In [ ]:
```
```

In [279…
```
# Fetch dataset
mushroom = fetch_ucirepo(id=73)
X = mushroom.data.features
y = mushroom.data.targets
```

In [280…
```
# Combine for processing convenience
df = pd.concat([y, X], axis=1)
df.replace('?', pd.NA, inplace=True)
```

In [281…
```
df.head()
```

Out[281…

	poisonous	cap-shape	cap-surface	cap-color	bruises	odor	gill-attachment	gill-spacing	gill-size	gill-color
0	p	x	s	n	t	p	f	c	n	k
1	e	x	s	y	t	a	f	c	b	k
2	e	b	s	w	t	l	f	c	b	n
3	p	x	y	w	t	p	f	c	n	n
4	e	x	s	g	f	n	f	w	b	k

5 rows × 23 columns

In [282…
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8124 entries, 0 to 8123
Data columns (total 23 columns):
 #   Column                    Non-Null Count  Dtype 
---  ------                    --------------  ----- 
 0   poisonous                 8124 non-null   object
 1   cap-shape                 8124 non-null   object
 2   cap-surface               8124 non-null   object
 3   cap-color                 8124 non-null   object
 4   bruises                   8124 non-null   object
 5   odor                      8124 non-null   object
 6   gill-attachment           8124 non-null   object
 7   gill-spacing              8124 non-null   object
 8   gill-size                 8124 non-null   object
 9   gill-color                8124 non-null   object
 10  stalk-shape               8124 non-null   object
 11  stalk-root                5644 non-null   object
 12  stalk-surface-above-ring  8124 non-null   object
 13  stalk-surface-below-ring  8124 non-null   object
 14  stalk-color-above-ring    8124 non-null   object
 15  stalk-color-below-ring    8124 non-null   object
 16  veil-type                 8124 non-null   object
 17  veil-color                8124 non-null   object
 18  ring-number               8124 non-null   object
 19  ring-type                 8124 non-null   object
 20  spore-print-color         8124 non-null   object
 21  population                8124 non-null   object
 22  habitat                   8124 non-null   object
dtypes: object(23)
memory usage: 1.4+ MB
```

In [283…
```
df.isnull().sum()
```

Out[283…
```
poisonous                    0
cap-shape                    0
cap-surface                  0
cap-color                    0
bruises                      0
odor                         0
gill-attachment              0
gill-spacing                 0
gill-size                    0
gill-color                   0
stalk-shape                  0
stalk-root                2480
stalk-surface-above-ring     0
stalk-surface-below-ring     0
stalk-color-above-ring       0
stalk-color-below-ring       0
veil-type                    0
veil-color                   0
ring-number                  0
ring-type                    0
spore-print-color            0
population                   0
habitat                      0
dtype: int64
```

In [284…
```
df['stalk-root'].value_counts()
```

Out[284…
```
stalk-root
b    3776
e    1120
c     556
r     192
Name: count, dtype: int64
```

In [285…
```
# First, replace '?' with actual NaN so pandas can recognize them
df.replace('?', pd.NA, inplace = True)

# Then, filter and show only rows with missing values
missing_rows = df[df.isna().any(axis=1)]
print(missing_rows)
```

```
     poisonous cap-shape cap-surface cap-color bruises odor gill-attachment
\
3984         e         x           y         b       t    n               f
4023         p         x           y         e       f    y               f
4076         e         f           y         u       f    n               f
4100         p         x           y         e       f    y               f
4104         p         x           y         n       f    f               f
...        ...       ...         ...       ...     ...  ...             ...
8119         e         k           s         n       f    n               a
8120         e         x           s         n       f    n               a
8121         e         f           s         n       f    n               a
8122         p         k           y         n       f    y               f
8123         e         x           s         n       f    n               a

     gill-spacing gill-size gill-color  ... stalk-surface-below-ring  \
3984            c         b          e  ...                        s
4023            c         n          b  ...                        s
4076            c         n          h  ...                        f
4100            c         n          b  ...                        s
4104            c         n          b  ...                        s
...           ...       ...        ...  ...                      ...
8119            c         b          y  ...                        s
8120            c         b          y  ...                        s
8121            c         b          n  ...                        s
8122            c         n          b  ...                        k
8123            c         b          y  ...                        s

     stalk-color-above-ring stalk-color-below-ring veil-type veil-color  \
3984                      e                      w         p          w
4023                      w                      w         p          w
4076                      w                      w         p          w
4100                      p                      p         p          w
4104                      p                      p         p          w
...                     ...                    ...       ...        ...
8119                      o                      o         p          o
8120                      o                      o         p          n
8121                      o                      o         p          o
8122                      w                      w         p          w
8123                      o                      o         p          o

     ring-number ring-type spore-print-color population habitat
3984           t         e                 w          c       w
4023           o         e                 w          v       p
4076           o         f                 h          y       d
4100           o         e                 w          v       d
4104           o         e                 w          v       l
...          ...       ...               ...        ...     ...
8119           o         p                 b          c       l
8120           o         p                 b          v       l
8121           o         p                 b          c       l
8122           o         e                 w          v       l
8123           o         p                 o          c       l

[2480 rows x 23 columns]
```

In [286…
```
# Fill missing values with mode for each column (safe assignment)
for col in df.columns:
    if df[col].isna().sum() > 0:
        df[col] = df[col].fillna(df[col].mode()[0])
```

In [287…
```
print(df.isna().sum())
```

```
poisonous                   0
cap-shape                   0
cap-surface                 0
cap-color                   0
bruises                     0
odor                        0
gill-attachment             0
gill-spacing                0
gill-size                   0
gill-color                  0
stalk-shape                 0
stalk-root                  0
stalk-surface-above-ring    0
stalk-surface-below-ring    0
stalk-color-above-ring      0
stalk-color-below-ring      0
veil-type                   0
veil-color                  0
ring-number                 0
ring-type                   0
spore-print-color           0
population                  0
habitat                     0
dtype: int64
```

In [288…
```
# want to see the class variable and categories
df['poisonous'].value_counts()
```

Out[288…
```
poisonous
e    4208
p    3916
Name: count, dtype: int64
```

In [289…
```
df['habitat'].value_counts()
```

Out[289…
```
habitat
d    3148
g    2148
p    1144
l     832
u     368
m     292
w     192
Name: count, dtype: int64
```

In [290…
```
df['stalk-root'].value_counts()
```

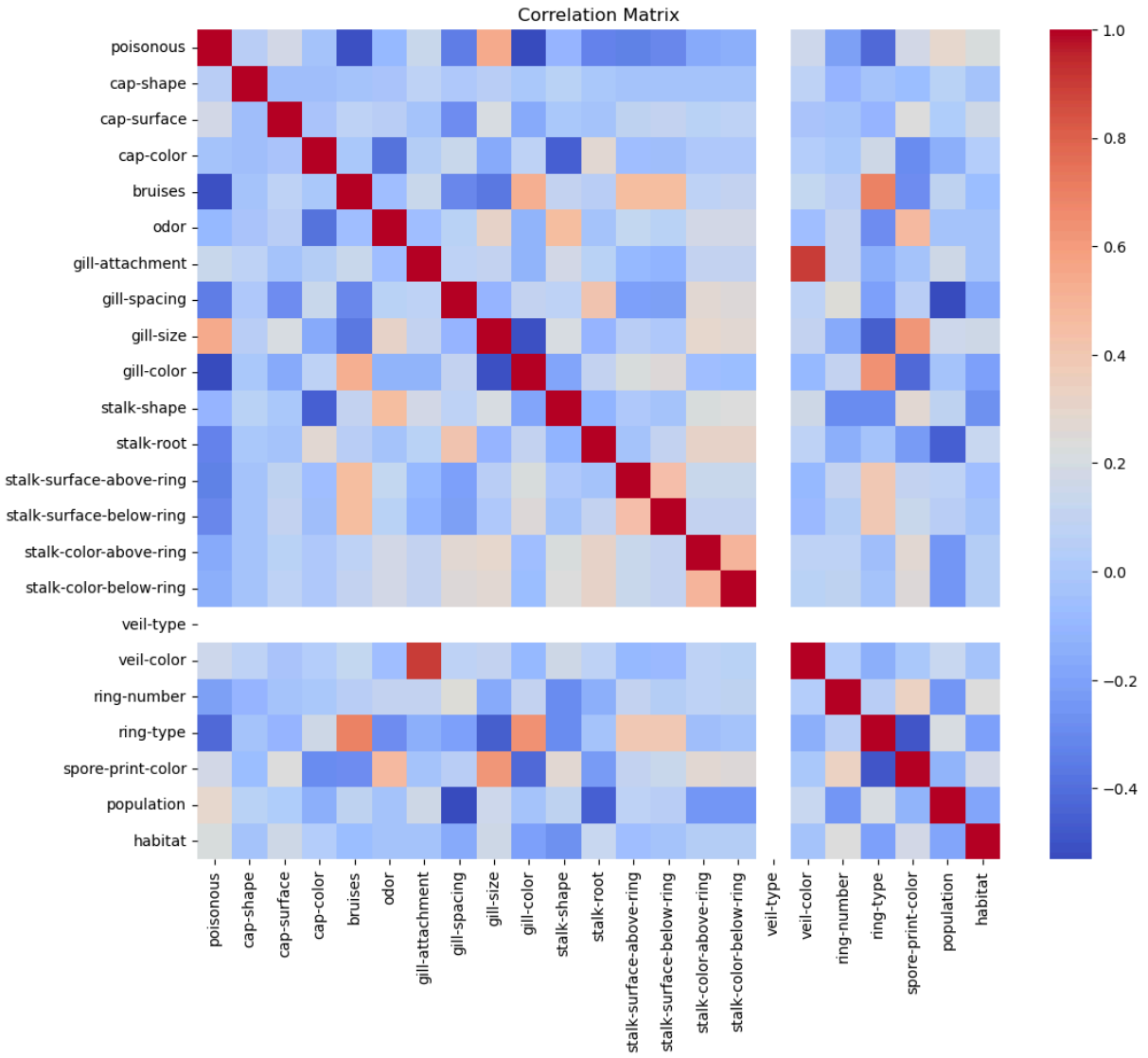
Out[290…
```
stalk-root
b    6256
e    1120
c     556
r     192
Name: count, dtype: int64
```

In [291…
```
# Label encoding
label_encoders = {}
for column in df.columns:
    le = LabelEncoder()
    df[column] = le.fit_transform(df[column])
    label_encoders[column] = le
```

In [292…
```
df['stalk-root'].value_counts()
```

Out[292…
```
stalk-root
0    6256
2    1120
1     556
3     192
Name: count, dtype: int64
```

In [293…
```
# Correlation heatmap
plt.figure(figsize=(12, 10))
sns.heatmap(df.corr(), annot=False, cmap='coolwarm')
plt.title("Correlation Matrix")
plt.show()
```



In [294…

```
# Target correlation ranking
print(df.corr()['poisonous'].sort_values(ascending=False))
```

```
poisonous                  1.000000
gill-size                  0.540024
population                 0.298686
habitat                    0.217179
cap-surface                0.178446
spore-print-color          0.171961
veil-color                 0.145142
gill-attachment            0.129200
cap-shape                  0.052951
cap-color                 -0.031384
odor                      -0.093552
stalk-shape               -0.102019
stalk-color-below-ring    -0.146730
stalk-color-above-ring    -0.154003
ring-number               -0.214366
stalk-surface-below-ring  -0.298801
stalk-root                -0.324194
stalk-surface-above-ring  -0.334593
gill-spacing              -0.348387
ring-type                 -0.411771
bruises                   -0.501530
gill-color                -0.530566
veil-type                       NaN
Name: poisonous, dtype: float64
```

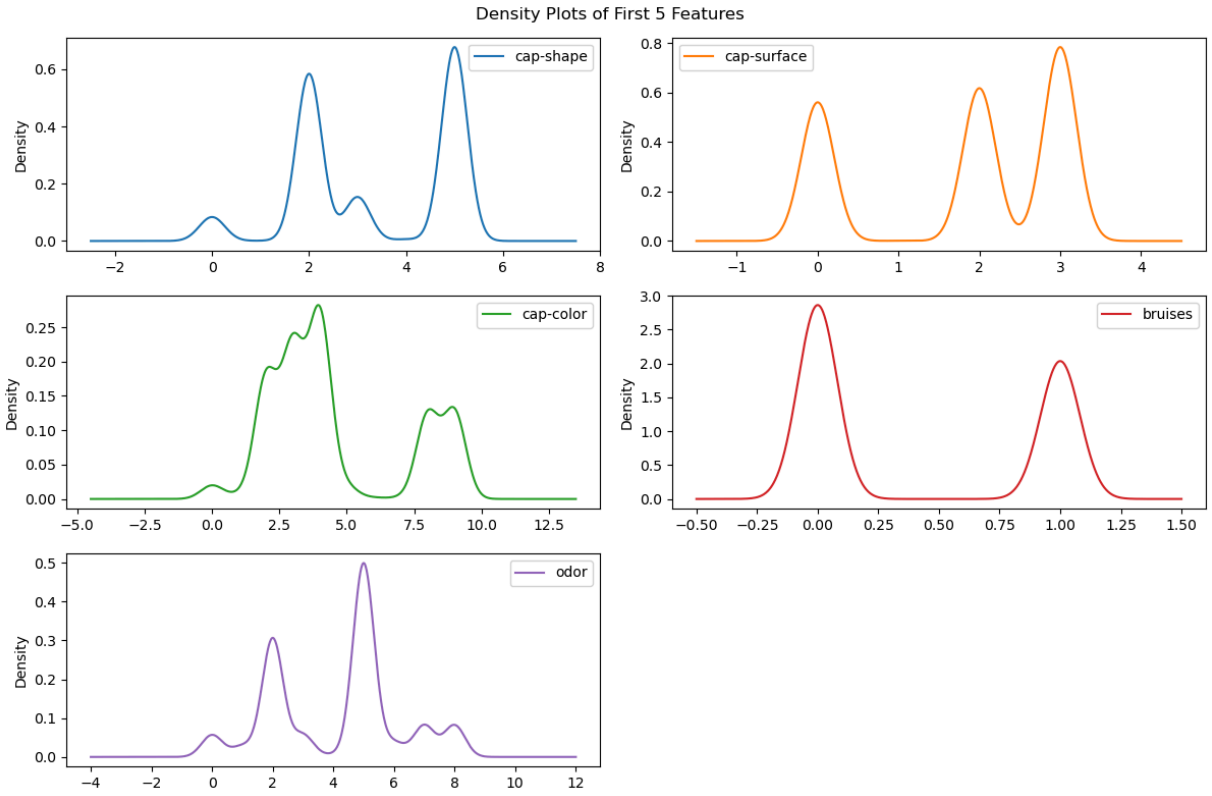
In [295…

```
# Outlier detection (Z-score method)
outliers = 0
for col in df.columns:
    z = (df[col] - df[col].mean()) / df[col].std()
    outliers += (np.abs(z) > 3).sum()
print(f"Total potential outliers across all features: {outliers}")
```

```
Total potential outliers across all features: 2102
```

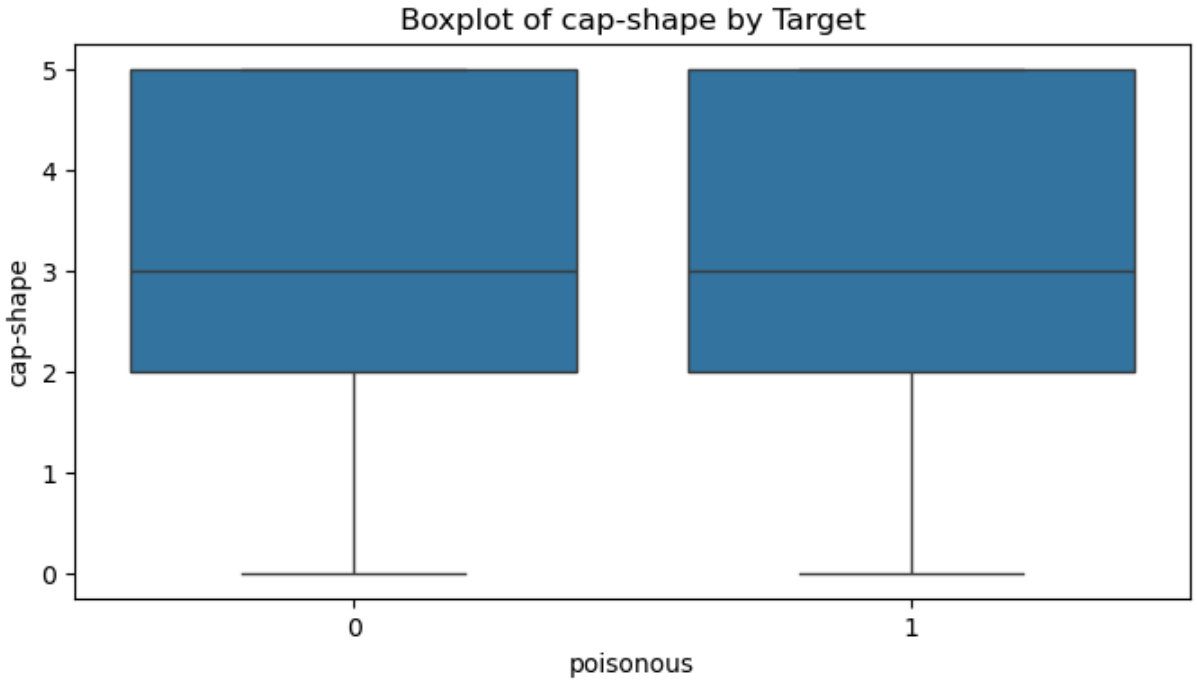
In [296…

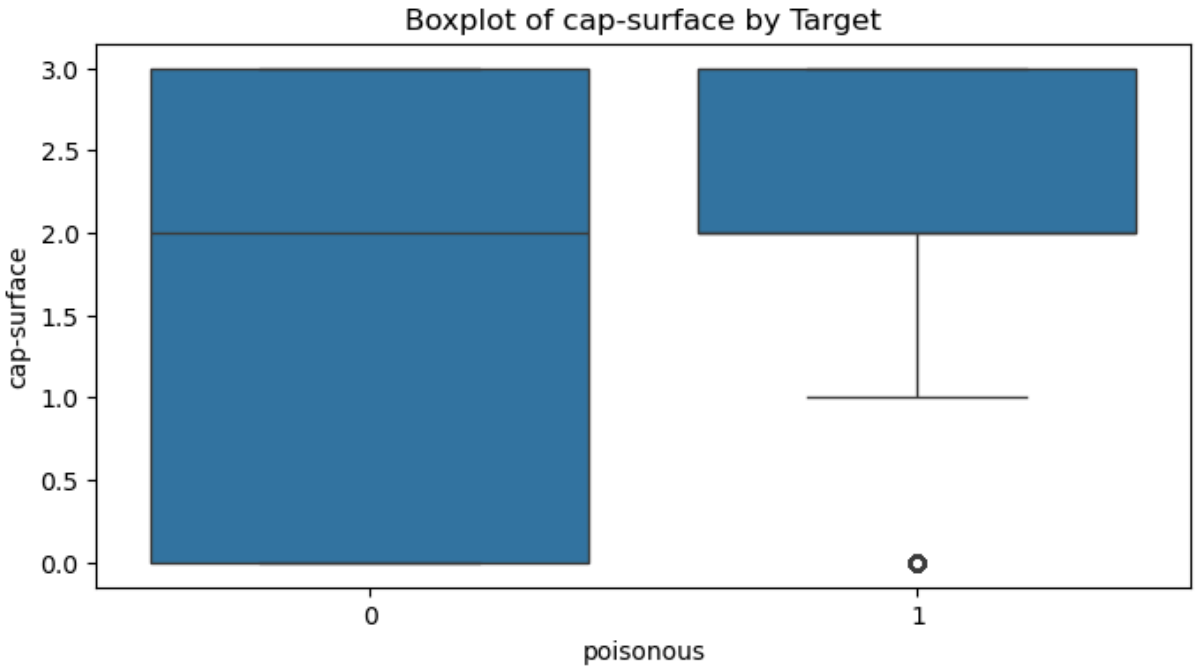
```
# Density plots of features
df.iloc[:, 1:6].plot(kind='density', subplots=True, layout=(3, 2), figsize=(
plt.suptitle('Density Plots of First 5 Features')
plt.tight_layout()
plt.show()
```

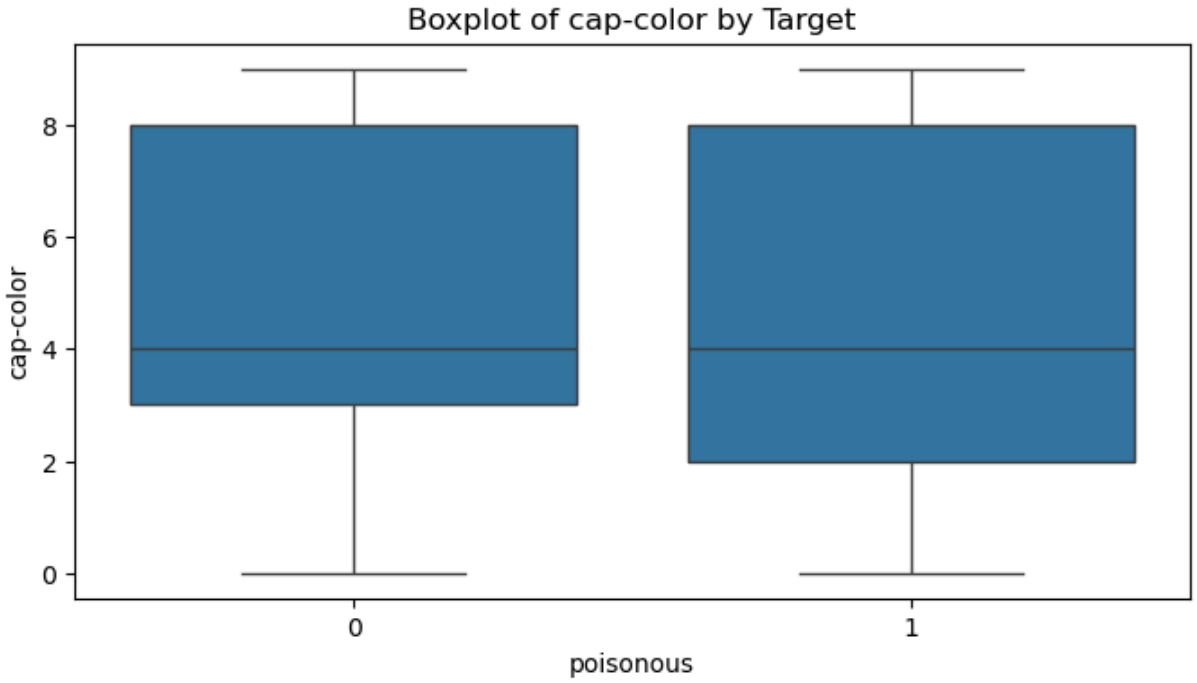


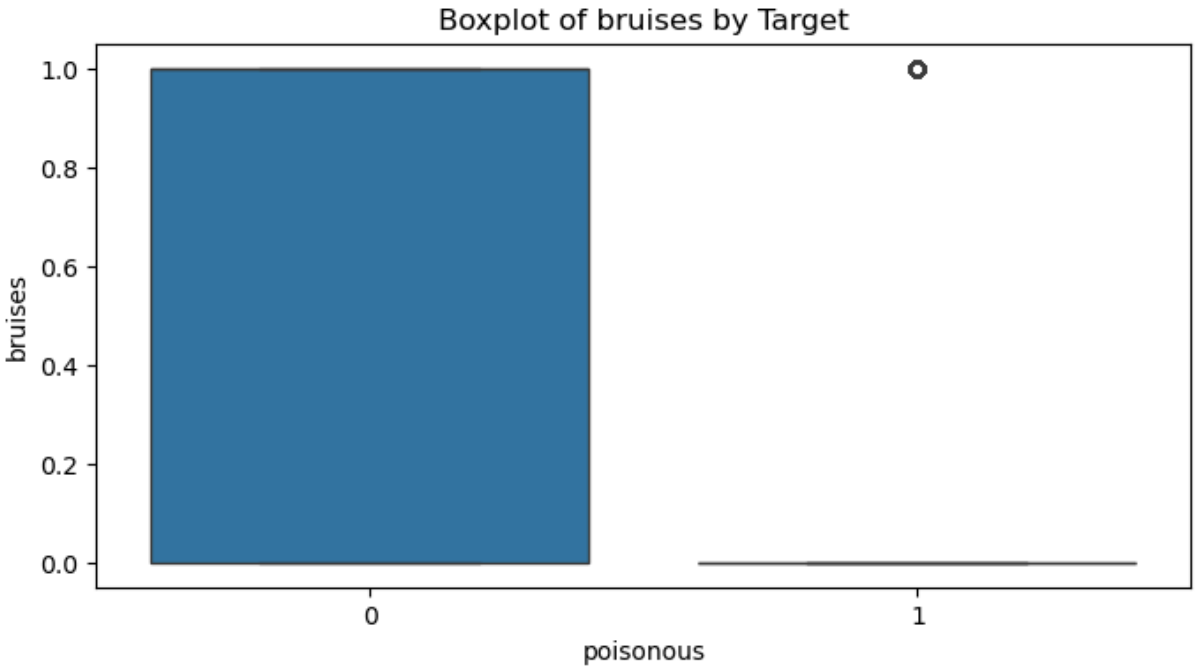
In [297…

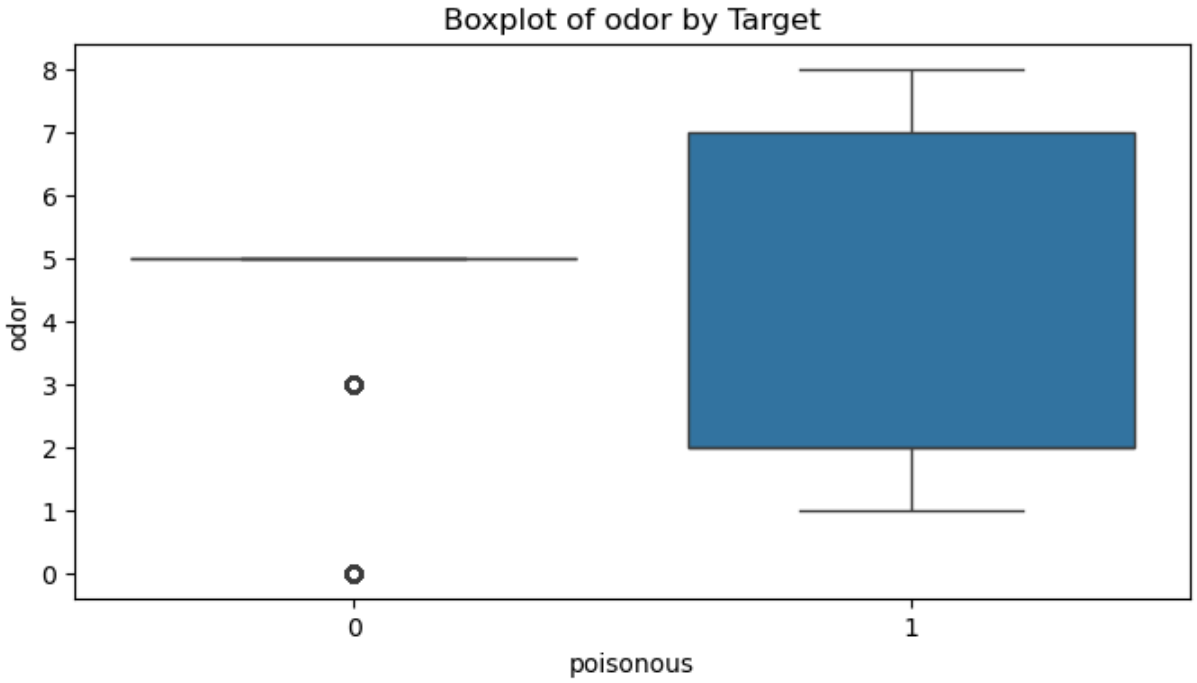
```
# Boxplots
for col in df.columns[1:6]:
    plt.figure(figsize=(8, 4))
    sns.boxplot(x='poisonous', y=col, data=df)
    plt.title(f'Boxplot of {col} by Target')
    plt.show()
```




Boxplot of cap-surface by Target
cap-surface
3.0
2.5
2.0
1.5
1.0
0.5
0.0
0
1
poisonous


Boxplot of cap-color by Target
cap-color
8
6
4
2
0
0
1
poisonous





In [298…
```
# Feature/Target
X = df.drop(columns=['poisonous'])
y = df['poisonous']
```

In [299…
```
# Train-test split
x_train, x_test, y_train, y_test = train_test_split(X, y, test_size=0.20, ra
```

In [300…
```
x_train.shape
```

Out[300… (6499, 22)

In [301…
```
x_test.shape
```

Out[301… (1625, 22)

In [302…
```
x_train.shape
```

Out[302… (6499, 22)

In [303…
```
y_test.shape
```

Out[303… (1625,)

In [304…
```
# ========================
# 2. Logistic Regression from Scratch
# ========================
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def compute_cost(X, y, weights):
    m = len(y)
    h = sigmoid(X.dot(weights))
    epsilon = 1e-5
    return (-1 / m) * (y.T.dot(np.log(h + epsilon)) + (1 - y).T.dot(np.log(1

def gradient_descent(X, y, weights, lr, iterations, tolerance=1e-6):
    cost_history = []
    for i in range(iterations):
        h = sigmoid(X.dot(weights))
        gradient = X.T.dot(h - y) / len(y)
        weights -= lr * gradient
        cost = compute_cost(X, y, weights)
        cost_history.append(cost)
        if i > 0 and abs(cost_history[-2] - cost_history[-1]) < tolerance:
            print(f"Converged at iteration {i}")
            break
    return weights, cost_history
```

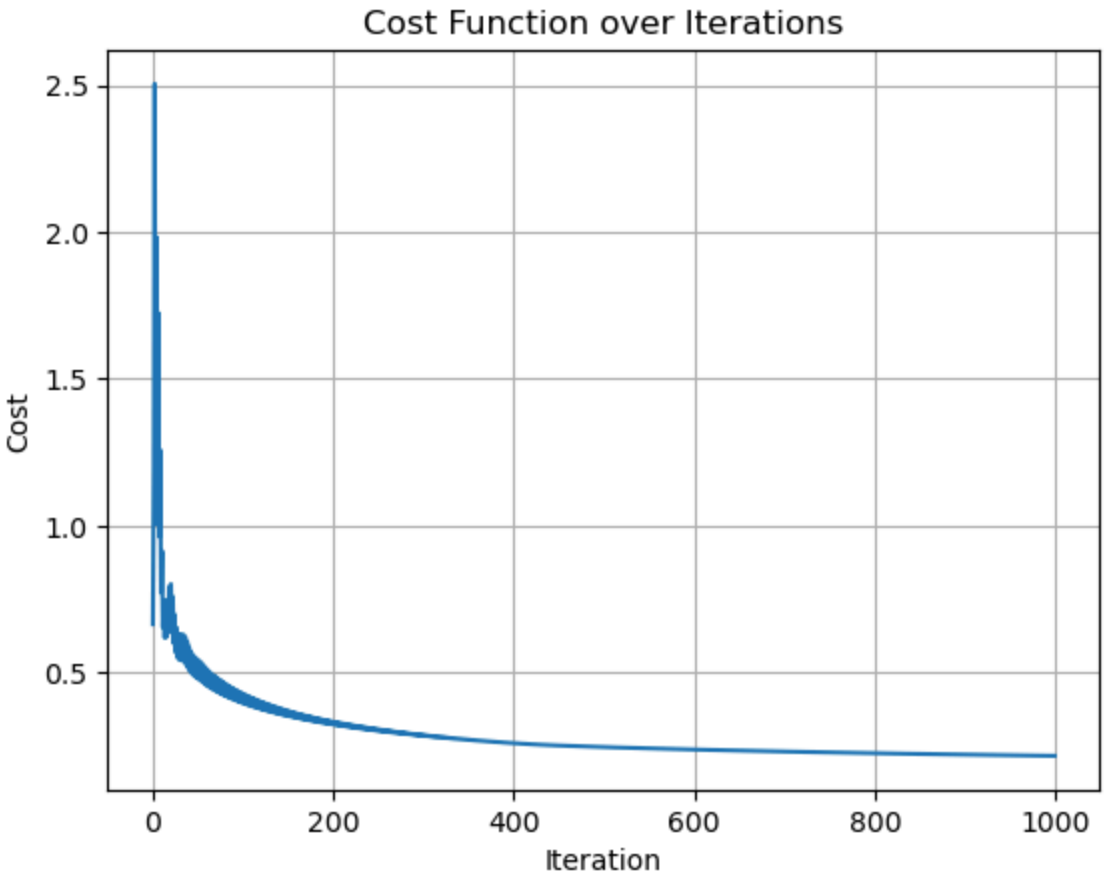
In [305…
```
# Add intercept term
X_train_bias = np.c_[np.ones((x_train.shape[0], 1)), x_train]
X_test_bias = np.c_[np.ones((x_test.shape[0], 1)), x_test]
```

In [306…
```
# Initialize
weights = np.zeros(X_train_bias.shape[1])
learning_rate = 0.1  # Chosen learning rate
iterations = 1000    # Max number of iterations
```

In [307…
```
# Train model
weights, cost_history = gradient_descent(X_train_bias, y_train, weights, lea
```

In [308…
```
# Cost plot
plt.plot(cost_history)
plt.title("Cost Function over Iterations")
plt.xlabel("Iteration")
plt.ylabel("Cost")
```

```
plt.grid(True)
plt.show()
```



In [309…

```
# Feature Weights
feature_weights = pd.Series(weights[1:], index=X.columns)
print("Feature Weights:")
print(feature_weights.sort_values(ascending=False))
```

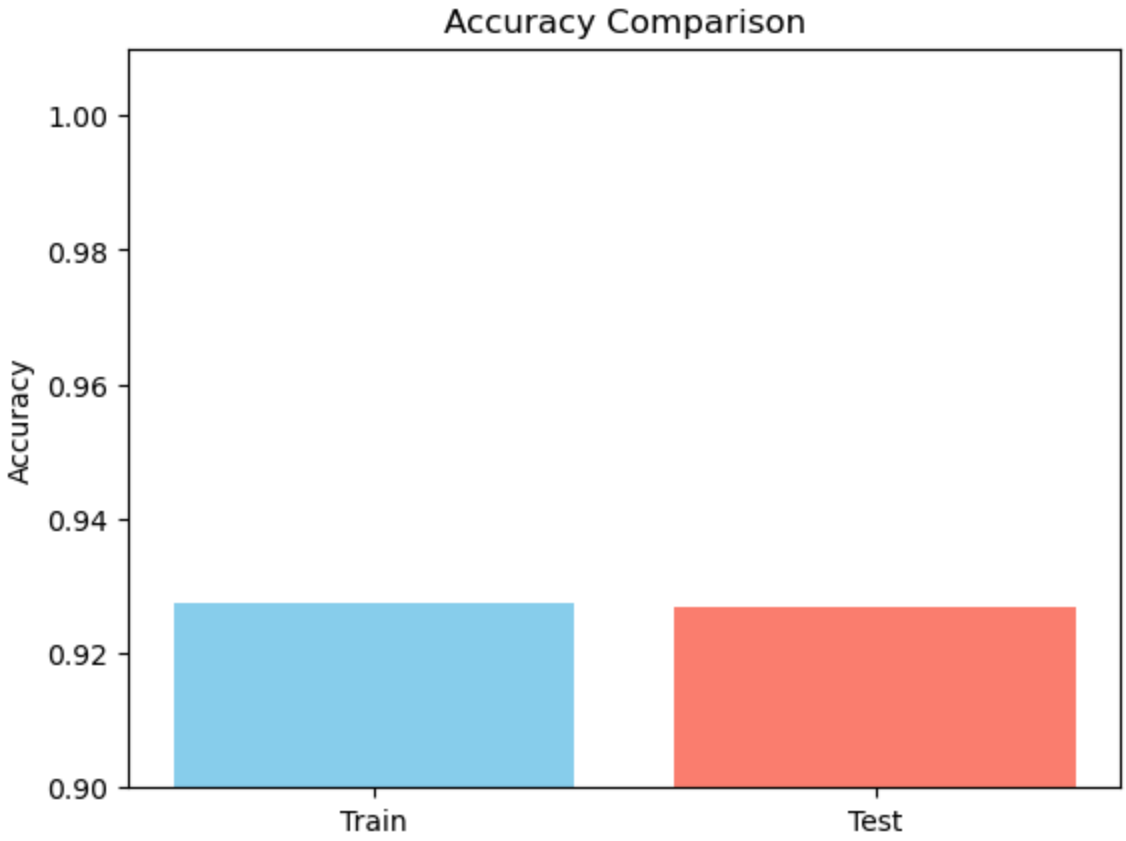
```
Feature Weights:
gill-size                  2.578724
veil-color                 1.037433
population                 0.473372
cap-surface                0.469103
gill-attachment            0.375727
habitat                    0.266160
cap-shape                  0.037160
stalk-color-below-ring     0.030625
cap-color                  0.013427
veil-type                  0.000000
ring-type                 -0.022715
stalk-color-above-ring    -0.054094
spore-print-color         -0.083653
gill-color                -0.159237
odor                      -0.167523
stalk-surface-below-ring  -0.348081
ring-number               -0.617421
stalk-shape               -0.902329
stalk-root                -0.975589
bruises                   -0.986027
stalk-surface-above-ring  -1.249071
gill-spacing              -1.327167
dtype: float64
```

In [310…
```python
# Predictions
train_probs = sigmoid(X_train_bias.dot(weights))
test_probs = sigmoid(X_test_bias.dot(weights))
y_train_pred = (train_probs >= 0.5).astype(int)
y_test_pred = (test_probs >= 0.5).astype(int)
```

In [311…
```python
# Accuracy
train_acc = accuracy_score(y_train, y_train_pred)
test_acc = accuracy_score(y_test, y_test_pred)
print(f"Train Accuracy: {train_acc:.4f} | Test Accuracy: {test_acc:.4f}")
```

```
Train Accuracy: 0.9275 | Test Accuracy: 0.9268
```

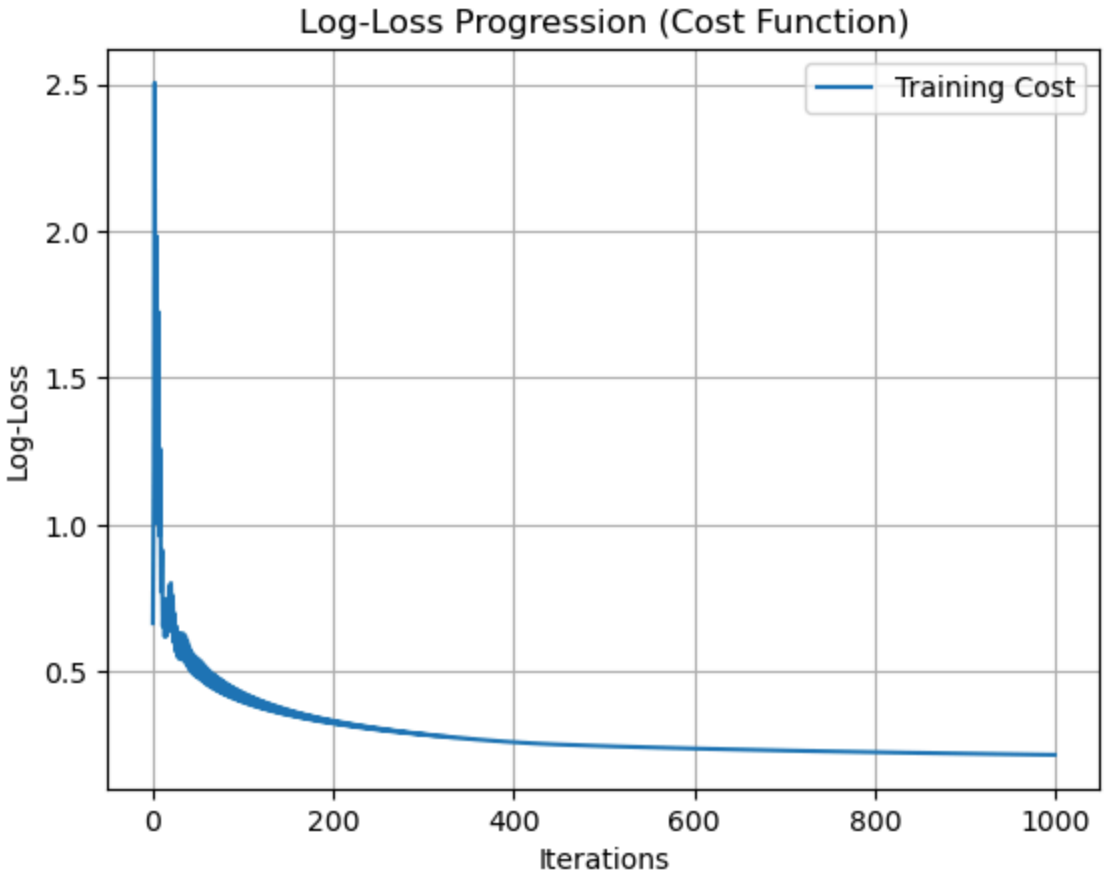
In [312…
```python
# Accuracy comparison bar plot
plt.bar(['Train', 'Test'], [train_acc, test_acc], color=['skyblue', 'salmon'
plt.ylim(0.9, 1.01)
plt.title('Accuracy Comparison')
plt.ylabel('Accuracy')
plt.show()
```



In [313…
```
# Log-loss
train_log_loss = log_loss(y_train, train_probs)
test_log_loss = log_loss(y_test, test_probs)
print(f"Train Log Loss: {train_log_loss:.4f}")
print(f"Test Log Loss: {test_log_loss:.4f}")
```

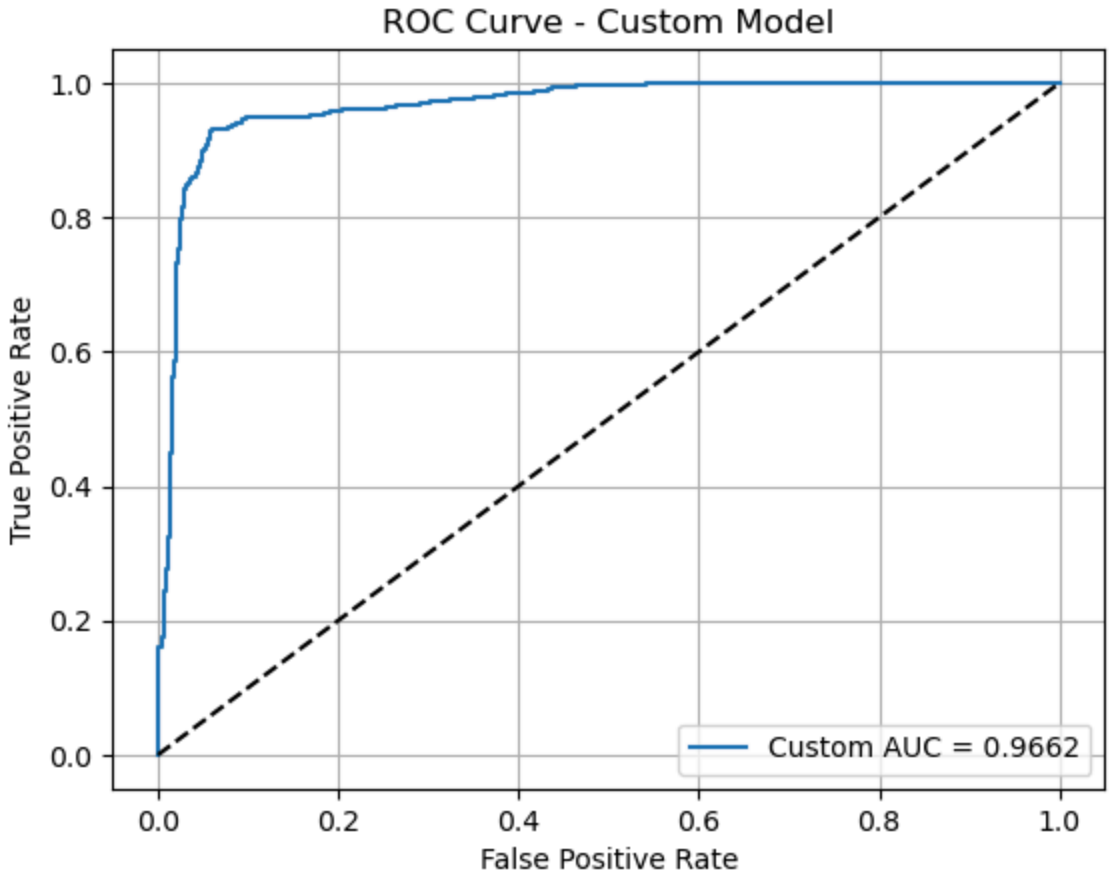
```
Train Log Loss: 0.2170
Test Log Loss: 0.2259
```

In [314…
```
# Log-loss curve
plt.plot(np.arange(len(cost_history)), cost_history, label='Training Cost')
plt.title('Log-Loss Progression (Cost Function)')
plt.xlabel('Iterations')
plt.ylabel('Log-Loss')
plt.legend()
plt.grid(True)
plt.show()
```



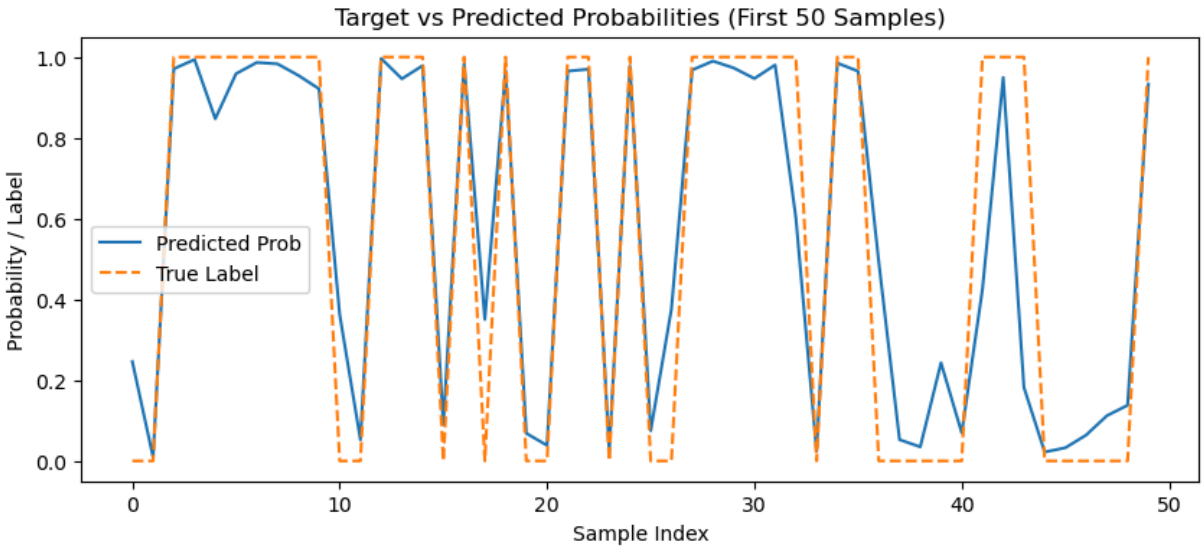
In [315…

```
# ROC Curve
fpr, tpr, _ = roc_curve(y_test, test_probs)
roc_auc = auc(fpr, tpr)
plt.plot(fpr, tpr, label=f"Custom AUC = {roc_auc:.4f}")
plt.plot([0, 1], [0, 1], 'k--')
plt.title("ROC Curve - Custom Model")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.legend()
plt.grid(True)
plt.show()
```



In [316…

```
# Target vs Prediction Plot (First 50)
plt.figure(figsize=(10, 4))
plt.plot(test_probs[:50], label='Predicted Prob')
plt.plot(y_test.values[:50], label='True Label', linestyle='--')
plt.legend()
plt.title("Target vs Predicted Probabilities (First 50 Samples)")
plt.xlabel("Sample Index")
plt.ylabel("Probability / Label")
plt.show()
```



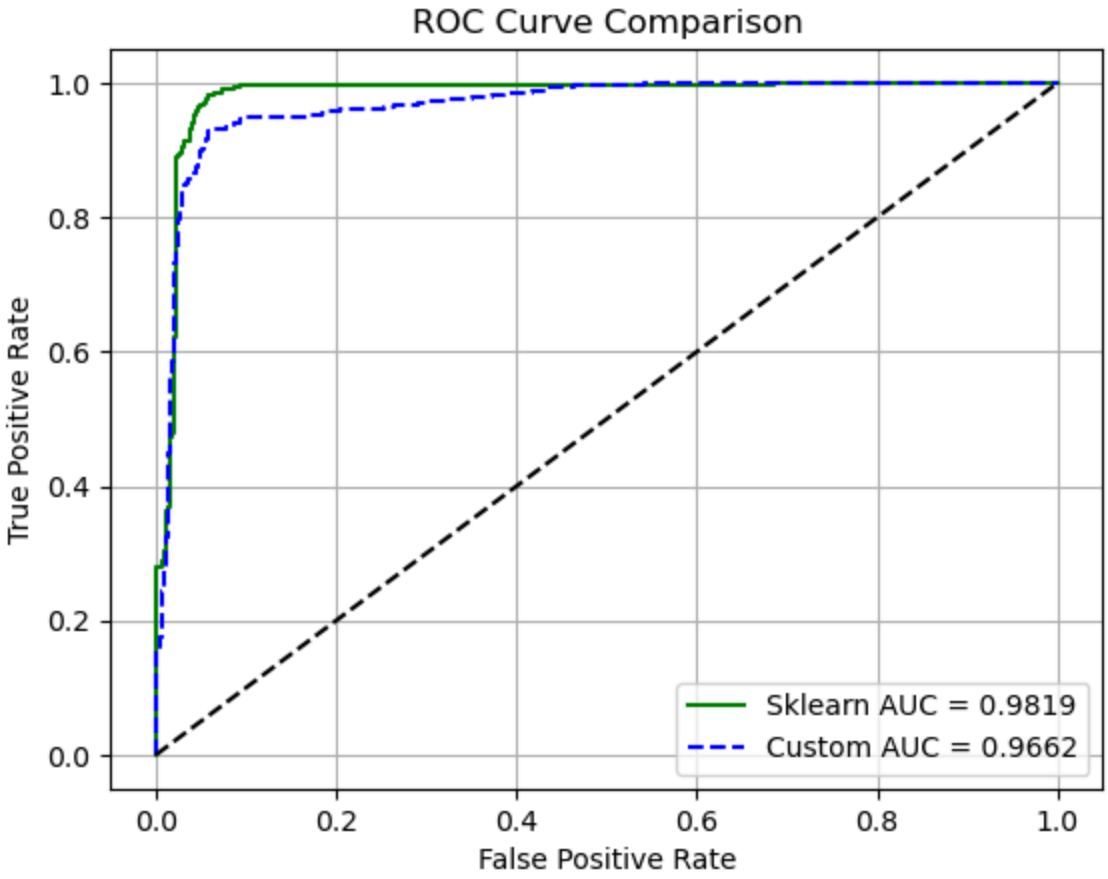
In [317…

```
# ========================
# 3. Comparison with Scikit-Learn
# ========================
model = LogisticRegression(max_iter=1000)
model.fit(x_train, y_train)

sk_train_probs = model.predict_proba(x_train)[:, 1]
sk_test_probs = model.predict_proba(x_test)[:, 1]
sk_y_train_pred = model.predict(x_train)
sk_y_test_pred = model.predict(x_test)

print("\n[Scikit-Learn Model Evaluation]")
print(f"Train Accuracy: {accuracy_score(y_train, sk_y_train_pred):.4f}")
print(f"Test Accuracy: {accuracy_score(y_test, sk_y_test_pred):.4f}")
print(f"Train Log Loss: {log_loss(y_train, sk_train_probs):.4f}")
print(f"Test Log Loss: {log_loss(y_test, sk_test_probs):.4f}")
```

```
[Scikit-Learn Model Evaluation]
Train Accuracy: 0.9598
Test Accuracy: 0.9545
Train Log Loss: 0.1413
Test Log Loss: 0.1589
```

In [318…

```
fpr_sk, tpr_sk, _ = roc_curve(y_test, sk_test_probs)
roc_auc_sk = auc(fpr_sk, tpr_sk)
plt.plot(fpr_sk, tpr_sk, label=f"Sklearn AUC = {roc_auc_sk:.4f}", color='gre
plt.plot(fpr, tpr, label=f"Custom AUC = {roc_auc:.4f}", linestyle='--', colo
plt.plot([0, 1], [0, 1], 'k--')
plt.title("ROC Curve Comparison")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.legend()
plt.grid(True)
plt.show()
```


ROC Curve Comparison
True Positive Rate
False Positive Rate
0.0
0.2
0.4
0.6
0.8
1.0
Sklearn AUC = 0.9819
Custom AUC = 0.9662